

Tab 1

Portfolio Project — Full Documentation (Frontend + Backend + CMS + 2-Week Plan)

Architecture Overview

This project is built using **two separate repositories** + a **custom-coded CMS** (not using existing headless CMS platforms). Everything — API, admin panel, content models — will be built from scratch.

◆ Repositories

1. **Frontend Repo** – React / Next.js + Tailwind CSS
 2. **Backend Repo (CMS + API)** – Python (Flask or FastAPI)
 3. **CMS** – Custom-built admin panel + content API + database
-

Repo Structure

1 Frontend Repo

Tech: React / Next.js + Tailwind CSS

Responsibilities:

- Fetch dynamic content from the custom CMS backend
- Render pages: Home, About, Projects, Skills, Experience, Blog, Contact
- SEO + animations + UI polish

Suggested Structure:

src/

components/

pages/ or app/

lib/api/

hooks/

layouts/

utils/

2 Backend Repo (Custom CMS + APIs)

Tech: Python Flask/FastAPI

DB: PostgreSQL / MongoDB / Supabase

Responsibilities:

- Full CMS implementation from scratch
 - Admin dashboard for managing content
 - CRUD APIs for each content type
 - Image/file uploads
 - Authentication for admin panel
 - Contact form handling
 - Serve data to frontend via REST API
-

Custom CMS (Built From Scratch)

You will create a minimal but complete CMS similar to Strapi/Sanity-like features.

CMS Features to Build

Admin Panel UI (React/Vite or Next.js)

- Login page (JWT-based auth)
- Dashboard
- CRUD pages for:
 - About
 - Skills
 - Projects
 - Blogs
 - Experience/Timeline
 - Testimonials

- Services
- Media upload system

Backend CMS APIs (Flask)

Auth APIs

- POST /auth/login
- POST /auth/refresh

Content APIs

- /about (GET, PUT)
- /skills (GET, POST, PUT, DELETE)
- /projects (GET, POST, PUT, DELETE)
- /blogs (GET, POST, PUT, DELETE)
- /experience (CRUD)
- /testimonials (CRUD)
- /services (CRUD)

Media Upload APIs

- POST /upload/image → save to file storage

Contact Form APIs

- POST /contact → Save message + send email



Database Schema (Custom)

Your CMS will store everything in your own database.

Example Tables

- users
- about
- skills
- projects
- blogs
- experience
- testimonials
- services
- messages (contact form)

- media
-

Data Flow

Frontend → Custom CMS API

Fetch content via REST API endpoints.

Admin Panel → CMS Backend

Full CRUD operations with authentication.

Contact Form Flow

Frontend → Backend → DB + Email

Benefits of This Architecture

- 100% customizable CMS built from scratch
 - Better control over security and logic
 - Reusable for any client portfolio or business website
 - No dependency on third-party CMS providers
-



2-Week Project Plan (Updated for CUSTOM CMS)

A complete revised plan to build CMS + Frontend portfolio.



Week 1 – Backend CMS + Database + Admin Panel Setup

Day 1 — Backend + Database Setup

- Initialize Backend Repo
- Setup Express project
- Connect database (Postgres/Mongo)
- Create basic folder structure

Day 2 — CMS Authentication System

- Create Admin user model
- Implement login API (JWT)
- Setup middleware for protected routes

Day 3 — CMS Content Models (DB Schemas)

- Create models for (About, Skills, Projects, Blogs, Experience, Testimonials, Services)
- Build CRUD APIs for each

Day 4 — File Upload System

- Setup multer / cloud storage
- Create /upload endpoint

Day 5 — Admin Panel (Frontend)

- Setup Admin Panel using React or Next.js
- Build Login page
- Build Dashboard

Day 6 — CMS CRUD Screens

- Add pages for managing:
 - About
 - Skills
 - Projects

Day 7 — Remaining CMS Pages

- Add pages for:
 - Blogs
 - Testimonials
 - Experience
 - Services
 - Fix UI
-



Week 2 – Portfolio Frontend, Integration & Deployment

Day 8 — Portfolio Frontend Setup

- Initialize Frontend Repo
- Setup Tailwind
- Build layout + home page

Day 9 — Fetch Content From CMS

- Connect to backend APIs
- Display About, Skills, Projects dynamically

Day 10 — Additional Sections

- Blog page
- Testimonials
- Experience timeline

Day 11 — Contact Form

- Build UI
- Connect POST /contact backend API

Day 12 — Deployment (Backend + CMS Admin)

- Deploy backend to Render/Railway
- Setup environment variables
- Deploy Admin Panel

Day 13 — Deploy Portfolio Frontend

- Deploy to Vercel/Netlify
- Configure domain + API URLs

Day 14 — Final Testing & Optimization

- API validation
 - Image optimizations
 - Security checks
 - Polish UI + fix bugs
-

Final Deliverables

Frontend Repo:

- Portfolio UI fully connected to custom CMS
- Fully responsive + SEO optimize
- CMS Frontend

Backend Repo (CMS):

- Custom admin panel
- REST APIs for all content models
- DB + authentication + file uploads
- Contact form save + email

Database:

- All tables for content management

Tab 2

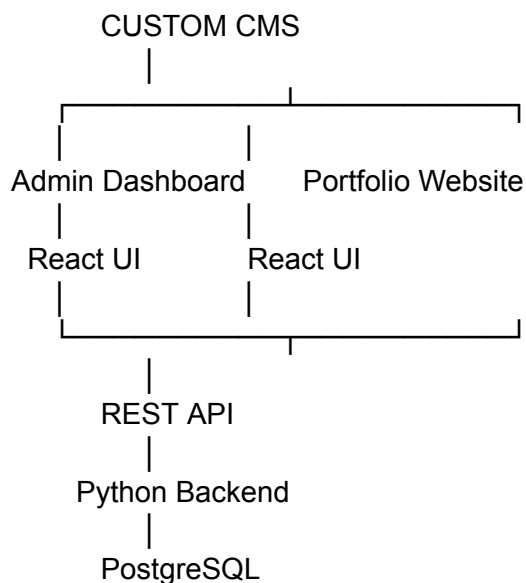
Python CMS Portfolio — Learning Resources

1. Understand the CMS Concept First

Before writing Python code, understand what you are actually building.

A CMS separates **website content** from **website presentation/code**.

Your project:



The portfolio itself is coded by you, but things such as:

- About information
- Skills
- Projects
- Experience
- Education
- Services
- Social links
- Resume
- Profile information
- Project images
- Featured projects

are stored in the database and managed through your **custom CMS**.

CMS Fundamentals

[Intro to CMS — Webflow University](#)

Even though this isn't a Python tutorial, it is useful because you need to understand concepts such as:

- Content types
- Content items
- Fields
- Collections
- Dynamic content
- Admin management
- Publishing

CMS Concept

[What is a CMS? — Optimizely](#)

2. Python Fundamentals

If you're already comfortable with Python, don't spend too much time here. Review only the concepts you'll use heavily in the project.

Python Full Course

[Python Full Course — freeCodeCamp](#)

Focus on:

- Functions
- Classes
- Objects
- Modules
- Packages
- Exceptions
- Decorators
- Virtual environments
- File handling
- Type hints

Official Python Documentation

[Python Documentation](#)

Python Tutorial

[Official Python Tutorial](#)

3. Django

For a CMS project, **Django is particularly valuable** because it already teaches you many concepts that CMS systems need.

Learn:

Django

- Project
- Apps
- Models
- Views
- URLs
- Forms
- Authentication
- Middleware
- ORM
- Admin

Django Full Course

[Django Full Course — freeCodeCamp](#)

Don't simply copy the tutorial.

Whenever the tutorial creates something like:

Blog

translate the concept into:

Portfolio CMS

Official Django Documentation

[Django Documentation](#)

 **Django Getting Started**

[Django Getting Started](#)

4. Django Models & ORM

This is one of the **most important parts** of your project.

Your CMS database might contain:

User
Profile
Project
Skill
Experience
Education
Service
SocialLink
Media

For example:

```
class Project(models.Model):
    title = models.CharField(max_length=200)
    description = models.TextField()
    github_url = models.URLField(blank=True)
    live_url = models.URLField(blank=True)
    featured = models.BooleanField(default=False)
    display_order = models.IntegerField(default=0)
```

 **Django Models & ORM**

[Django ORM Tutorial — Very Academy](#)

 **Django Models**

[Django Models Documentation](#)

 **Django QuerySets**

[Django QuerySet API](#)

Learn:

```
objects.create()
objects.get()
objects.filter()
objects.exclude()
objects.update()
objects.delete()
order_by()
select_related()
prefetch_related()
```

These will be used heavily in your CMS.

5. PostgreSQL

Your content should be stored independently from your frontend.

Architecture:

```
React
↓
Django REST API
↓
Django ORM
↓
PostgreSQL
```



PostgreSQL Full Course

[PostgreSQL Full Course — freeCodeCamp](#)



PostgreSQL Tutorial

[PostgreSQL Official Tutorial](#)

Focus on:

- Tables
- Primary keys

- Foreign keys
 - Relationships
 - Constraints
 - JOIN
 - Indexes
 - Transactions
 - SQL queries
-

6. Django REST Framework

This is **probably the most important resource after Django** for your project.

Your React frontend will communicate with Python through REST APIs.

For example:

GET /api/projects/
GET /api/projects/1/

POST /api/projects/

PUT /api/projects/1/

DELETE /api/projects/1/

Django REST Framework

[Django REST Framework Tutorial — Very Academy](#)

Official DRF Documentation

[Django REST Framework Documentation](#)

DRF Quickstart

[DRF Quickstart](#)

Learn:

Serializer
ModelSerializer
APIView

ViewSet
Router
Request
Response
Permissions
Authentication
Pagination
Filtering

7. CRUD — Core of Your CMS

CRUD is the foundation of your CMS.

C → Create
R → Read
U → Update
D → Delete

For example:

Projects

POST /api/projects/
GET /api/projects/
GET /api/projects/{id}/
PUT /api/projects/{id}/
DELETE /api/projects/{id}/

Skills

POST /api/skills/
GET /api/skills/
PUT /api/skills/{id}/
DELETE /api/skills/{id}/

Experience

POST /api/experience/
GET /api/experience/
PUT /api/experience/{id}/
DELETE /api/experience/{id}/

 **Django REST CRUD**



8. Authentication & Authorization

Your CMS needs two sides:

PUBLIC



Read published portfolio content

ADMIN



Login



CMS Dashboard



Create / Update / Delete / Publish

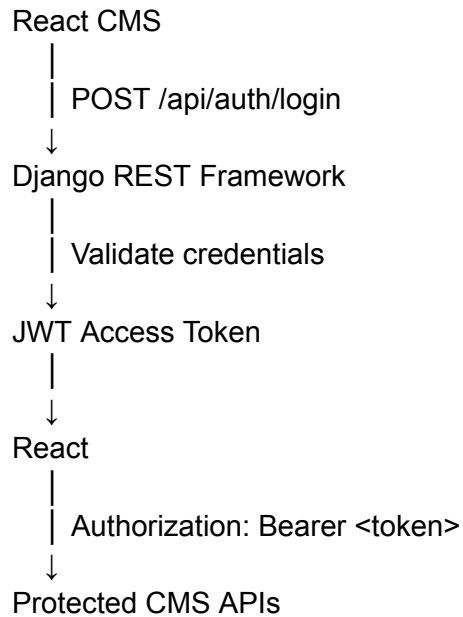
You should understand:

- Authentication
 - Authorization
 - User
 - Admin
 - Permissions
 - JWT
 - Access token
 - Refresh token
-

9. JWT Authentication with Django REST Framework

For your React CMS, JWT is a good concept to learn.

Architecture:



JWT Authentication with Django REST Framework

[DRF JWT Authentication — Code With Stein](#)

Simple JWT Documentation

[Django REST Framework Simple JWT](#)

DRF Authentication

[DRF Authentication Documentation](#)

Learn:

- Access Token
- Refresh Token
- Bearer Token
- Token Expiration
- Authentication Classes
- Permissions

10. Django Permissions

Your CMS should not allow a normal visitor to modify your portfolio.

For example:

GET /api/projects/



PUBLIC → Allowed

POST /api/projects/



PUBLIC → Denied

ADMIN → Allowed



DRF Permissions

[DRF Permissions Documentation](#)

Learn:

- AllowAny
- IsAuthenticated
- IsAdminUser
- Custom permissions

For your project, you can eventually create:

IsPortfolioAdmin

11. React + Django REST API

Your project should have separate frontend/backend repositories:

portfolio-frontend/

portfolio-backend/

The frontend communicates with the backend:

React



HTTP



Django REST API

|
↓
PostgreSQL

React + Django

[React + Django Full Stack Tutorial — freeCodeCamp](#)

The important thing isn't the exact project in the video. Focus on understanding:

React
↓
API request
↓
Django
↓
Database
↓
JSON response
↓
React

12. Axios

For your React CMS, you can use Axios for API communication.

Example:

```
const response = await axios.get(
  "http://localhost:8000/api/projects/"
);
```

Authenticated request:

```
const response = await axios.post(
  "/api/projects/",
  projectData,
  {
    headers: {
      Authorization: `Bearer ${accessToken}`
    }
  }
);
```

13. File & Image Upload

Your CMS should ideally allow the administrator to upload:

- Profile Photo
- Project Images
- Company Logos
- Certificates
- Resume
- Other Portfolio Media

Django provides:

- FileField
- ImageField
- MEDIA_ROOT
- MEDIA_URL

Django File Uploads

[Django File Uploads Documentation](#)

Managing Files

[Django Managing Files](#)

Learn:

- multipart/form-data
- File validation
- Image validation
- File size limits
- File storage
- Media URLs

14. CMS Content Structure

This is where you should start thinking like a **CMS developer**, rather than simply a Python developer.

Your database could eventually look like:

User

- Admin

Profile

- name
- title
- bio
- profile_image
- resume
- contact_information

Projects

- title
- description
- image
- github_url
- live_url
- featured
- display_order

Skills

- name
- category
- proficiency
- display_order

Experience

- company
- position
- description
- start_date
- end_date

Education

- institution
- degree

- description
- year

Services

- title
- description
- icon

SocialLinks

- platform
- url
- display_order

This is one of the most important design exercises for your project.

15. Draft & Published Content

This is a feature that will make your project feel more like a **real CMS**.

Instead of immediately showing every change:

Admin

↓

Edit Project

↓

Save as Draft

↓

Preview

↓

Publish

↓

Portfolio displays it

For example:

```
class Project(models.Model):  
  
    STATUS_CHOICES = [  
        ("draft", "Draft"),  
        ("published", "Published"),  
    ]
```

```
status = models.CharField(  
    max_length=20,  
    choices=STATUS_CHOICES,  
    default="draft"  
)
```

Then:

GET /api/projects/

for the public website should return only:

status = published

while the CMS can retrieve both:

draft

published

This is an important CMS concept to implement yourself.

16. Django Admin vs Your Custom CMS

Important for your project:

Do **not** confuse Django Admin with the CMS you're building.

Django provides:

Django Admin

out of the box.

But your project requirement is:

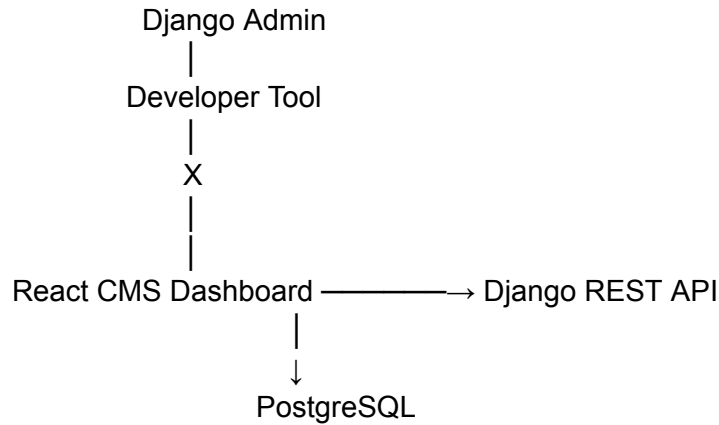
Your own CMS Dashboard

Therefore:

 Don't rely on Django Admin as the final CMS UI

 Build your own React CMS dashboard

Your architecture should be:



You **can** use Django Admin during development to inspect database records, but the actual portfolio content management interface should be your own implementation.

17. API Documentation

Once your APIs are ready, document them using OpenAPI/Swagger.

For example:

Authentication

POST /api/auth/login

POST /api/auth/refresh

Projects

GET /api/projects

POST /api/projects

GET /api/projects/{id}

PUT /api/projects/{id}

DELETE /api/projects/{id}

Skills

GET /api/skills

POST /api/skills

PUT /api/skills/{id}

DELETE /api/skills/{id}

DRF OpenAPI / Schema

[DRF Schemas Documentation](#)

drf-spectacular

[drf-spectacular Documentation](#)

18. Testing

Once your CMS works, learn to test:

Authentication

CRUD APIs

Permissions

Database operations

File uploads

Validation

Pytest

[Pytest Tutorial — freeCodeCamp](#)

Django Testing

[Django Testing Documentation](#)

DRF Testing

[DRF Testing Documentation](#)

19. Complete Resource Order

If you're learning specifically for this project, I recommend following this order:

Phase 1 — CMS Concept

1. [What is a CMS?](#)

2. [Intro to CMS](#)

Learn:

Content
Content Type
Content Item
Fields
Media
Draft
Published
Admin

Phase 2 — Python

3. [Python Full Course](#)
4. [Python Documentation](#)

Phase 3 — Django

5. [Django Full Course](#)
6. [Django Documentation](#)

Phase 4 — Database

7. [PostgreSQL Full Course](#)
8. [PostgreSQL Tutorial](#)

Phase 5 — API

9. [Django REST Framework Tutorial](#)
10. [DRF Documentation](#)

Phase 6 — Authentication

11. [JWT Authentication Tutorial](#)
12. [Simple JWT Documentation](#)

Phase 7 — React Integration

13. [React Documentation](#)
14. [Axios Documentation](#)

Phase 8 — Media

15. [Django File Upload Documentation](#)

Phase 9 — Testing

16. [Django Testing](#)

17. [Pytest Documentation](#)

What you should actually learn for this project

You **don't need to master all of Django** before starting.

Your target knowledge should be:

Python
↓
Django
↓
Django Models
↓
Django ORM
↓
PostgreSQL
↓
Django REST Framework
↓
Serializers
↓
ViewSets / APIs
↓
JWT Authentication
↓
Permissions
↓
File Upload
↓
React API Integration
↓
Custom CMS

And the final project should look like:



